Name: M.Divya
Reg. No: 192373046

DSA1604 – Knowledge Engineering and Data Science

Experiment 1: Knowledge Representation Using Python

Aim

To represent facts about a student using a Python dictionary and display the stored knowledge.

Software Required

Python 3.x (IDLE / any Python IDE)

Algorithm

1. **Start the program.** Decide which facts about the student need to be stored: name, department, course and year.
2. **Create a dictionary named student.** Each fact is stored as a key–value pair, for example "name": "Arun", so every attribute can be looked up by its key.
3. **Print the heading.** Display "Student Knowledge Representation" so the output is clearly labelled.
4. **Traverse the dictionary.** Use the items() method inside a for loop, which returns each key together with its value one pair at a time.
5. **Display each fact.** In every iteration print the key, a colon and the value, so each attribute appears on its own line.
6. **Stop the program.** The loop ends automatically after the last key–value pair has been displayed.

Pseudocode

```
START
  CREATE dictionary student
    name <- "Arun"
    department <- "CSE"
    course <- "Python Programming"
    year <- 2
  DISPLAY "Student Knowledge Representation"
  FOR EACH (key, value) IN student
    DISPLAY key, ":", value
  END FOR
STOP
```

Program

```
student = {
    "name": "Arun",
    "department": "CSE",
    "course": "Python Programming",
    "year": 2
}

print("Student Knowledge Representation")
for key, value in student.items():
    print(key, ":", value)
```

Output Screenshot

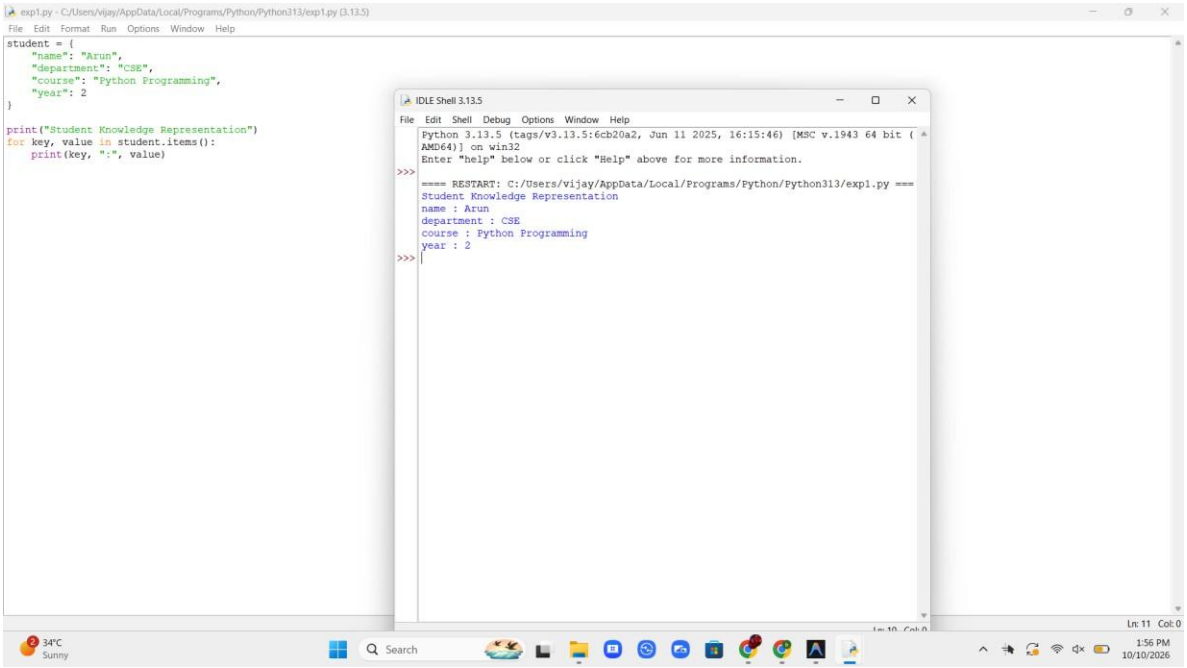


Fig 1. Output of Experiment 1

Result

Knowledge was represented as structured facts using a Python dictionary and displayed successfully.

Experiment 2: Knowledge Acquisition

Aim

To acquire knowledge from a user and store it as structured facts.

Software Required

Python 3.x (IDLE / any Python IDE)

Algorithm

1. **Start the program.** Plan to collect three facts about an expert: name, area of expertise and years of experience.
2. **Read the expert's name.** Use input() to accept the name from the user as a string and store it in the variable name.
3. **Read the area of expertise.** Use input() again and store the domain as a string in the variable domain.
4. **Read the years of experience.** Accept the value with input() and convert it to an integer using int(), so it can be used in calculations later.
5. **Store the facts in a dictionary.** Create a dictionary named knowledge with the keys "expert", "domain" and "experience" holding the three values.
6. **Display the acquired knowledge.** Print each fact by reading it back from the dictionary using its key.
7. **Stop the program.**

Pseudocode

```
START
   READ name
   READ domain
   READ experience
   CONVERT experience TO integer
   CREATE dictionary knowledge
      expert <- name
      domain <- domain
      experience <- experience
   DISPLAY "Acquired Knowledge"
   DISPLAY "Expert:", knowledge["expert"]
   DISPLAY "Domain:", knowledge["domain"]
   DISPLAY "Experience:", knowledge["experience"], "years"
STOP
```

Program

```
name = input("Enter expert name: ")
domain = input("Enter area of expertise: ")
experience = int(input("Enter years of experience: "))

knowledge = {
    "expert": name,
    "domain": domain,
    "experience": experience
}

print("\nAcquired Knowledge")
print("Expert:", knowledge["expert"])
print("Domain:", knowledge["domain"])
print("Experience:", knowledge["experience"], "years")
```

Output Screenshot

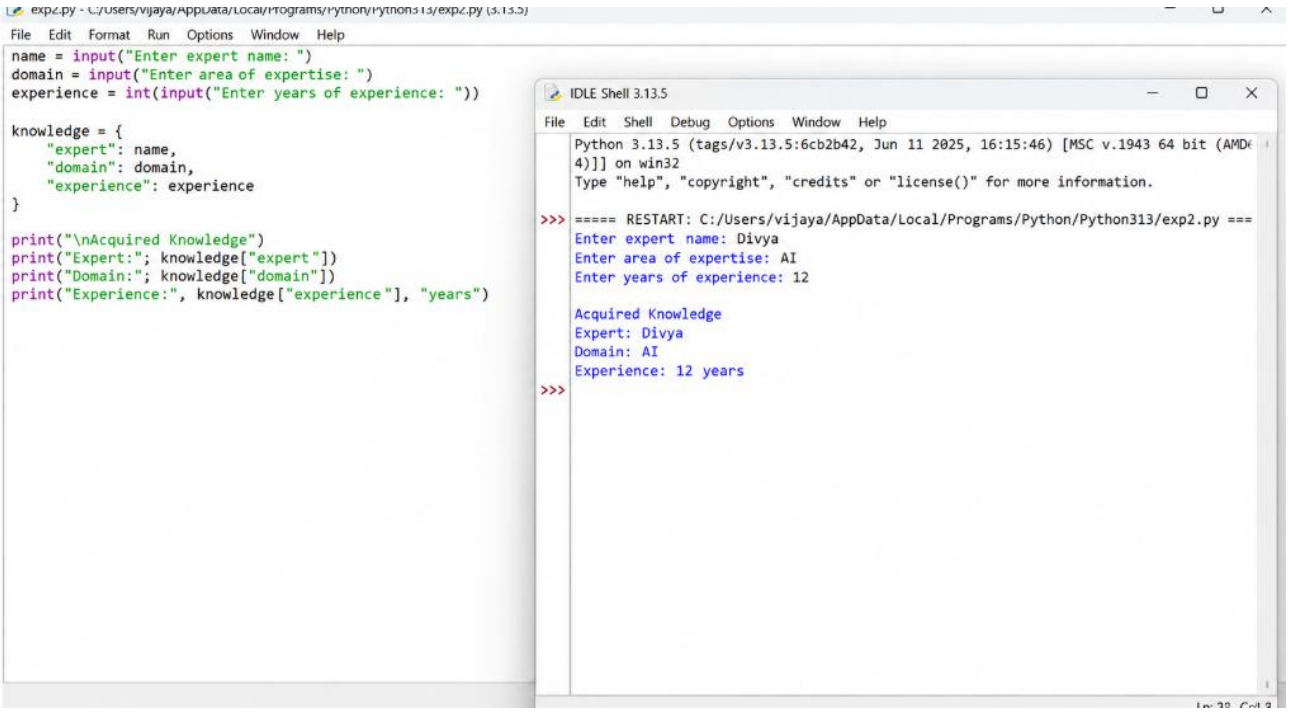


Fig 2. Output of Experiment 2

Result

Knowledge was collected from the user and stored in a structured form.

Experiment 3: Knowledge-Based Course Recommendation System

Aim

To recommend a course using a small knowledge base and decision rules.

Software Required

Python 3.x (IDLE / any Python IDE)

Algorithm

1. **Start the program.** The knowledge base consists of simple rules that link an area of interest to a course.
2. **Read the student's interest.** Accept the interest (AI, Data or Web) using input(), then apply strip() and lower() to remove extra spaces and make the comparison case-insensitive.
3. **Apply rule 1.** If the interest is "ai", set the course to "Artificial Intelligence".
4. **Apply rule 2.** Otherwise, if the interest is "data", set the course to "Data Science".
5. **Apply rule 3.** Otherwise, if the interest is "web", set the course to "Web Development".
6. **Apply the fallback rule.** If none of the rules match, set the course to "No matching course found".
7. **Display the recommendation.** Print the selected course and stop the program.

Pseudocode

```
START
   READ interest
   interest <- LOWERCASE(TRIM(interest))
   IF interest = "ai" THEN
      course <- "Artificial Intelligence"
   ELSE IF interest = "data" THEN
      course <- "Data Science"
   ELSE IF interest = "web" THEN
      course <- "Web Development"
   ELSE
      course <- "No matching course found"
   END IF
   DISPLAY "Recommended Course:", course
STOP
```

Program

```
interest = input("Enter your interest (AI, Data, Web): ").strip().lower()

if interest == "ai":
    course = "Artificial Intelligence"
elif interest == "data":
    course = "Data Science"
elif interest == "web":
    course = "Web Development"
else:
    course = "No matching course found"

print("Recommended Course:", course)
```

Output Screenshot

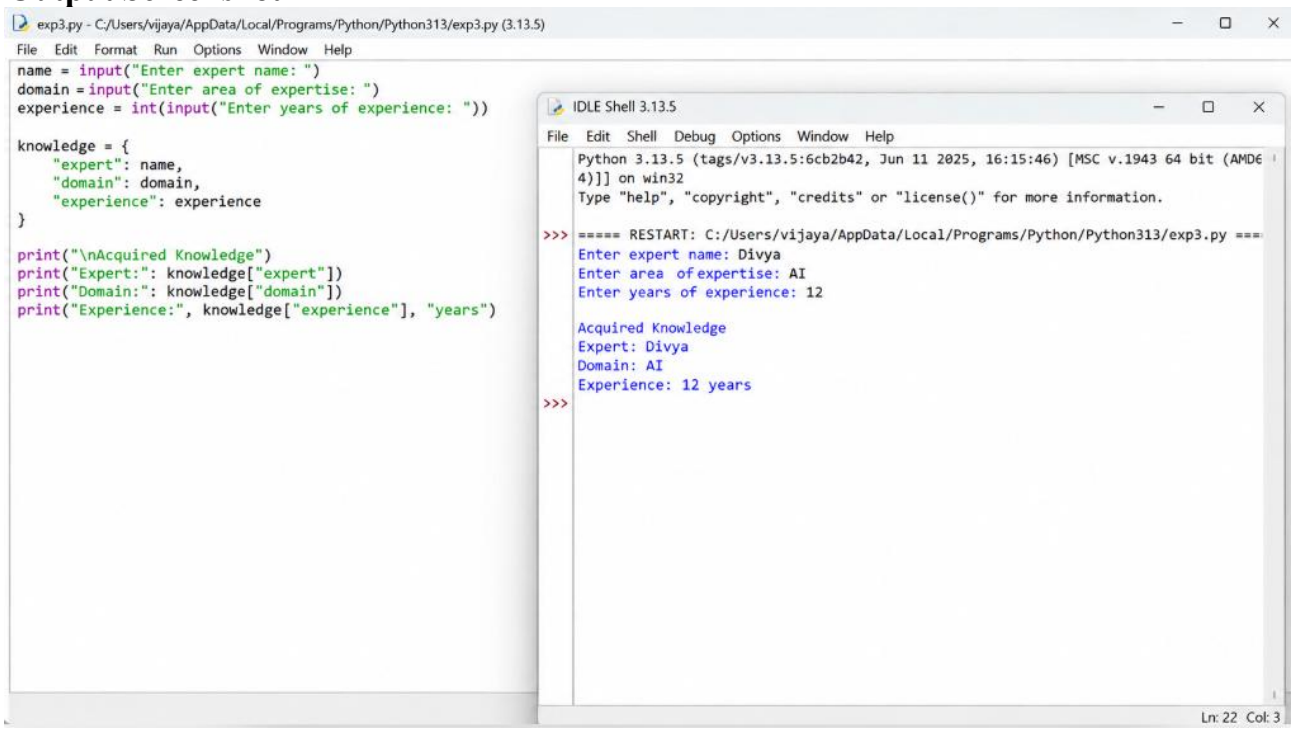


Fig 3. Output of Experiment 3

Result

The system used stored knowledge and rules to produce a course recommendation.

Experiment 4: Rule-Based Expert System for Network Troubleshooting

Aim

To build a rule-based expert system that infers a network troubleshooting action from the symptoms given by the user.

Software Required

Python 3.x (IDLE / any Python IDE)

Algorithm

1. **Start the program.** The knowledge base consists of IF-THEN rules that link network symptoms to a fault and its remedy.
2. **Read the symptoms.** Use input() to ask four yes/no questions: router power light, Wi-Fi connection, ping result and website access. Apply strip() and lower() to each answer.
3. **Apply rule 1.** If the router light is off, conclude that the router has no power.
4. **Apply rule 2.** Otherwise, if Wi-Fi is not connected, conclude that the device connection must be fixed.
5. **Apply rule 3.** Otherwise, if the ping fails, conclude that there is no internet from the service provider.
6. **Apply rule 4.** Otherwise, if websites do not open, conclude that there is a DNS problem.
7. **Apply the default rule.** If no fault rule fires, conclude that the network is working normally.
8. **Display the diagnosis and stop.** Print the advice selected by the rule that fired.

Pseudocode

```
START
    READ router_light, wifi, ping, website
    IF router_light = "no" THEN
        advice <- "Check power cable and adapter"
    ELSE IF wifi = "no" THEN
        advice <- "Turn Wi-Fi on and re-enter password"
    ELSE IF ping = "no" THEN
        advice <- "Restart router or call the provider"
    ELSE IF website = "no" THEN
        advice <- "Change the DNS server"
    ELSE
        advice <- "Network is working normally"
    END IF
    DISPLAY "Diagnosis:", advice
STOP
```

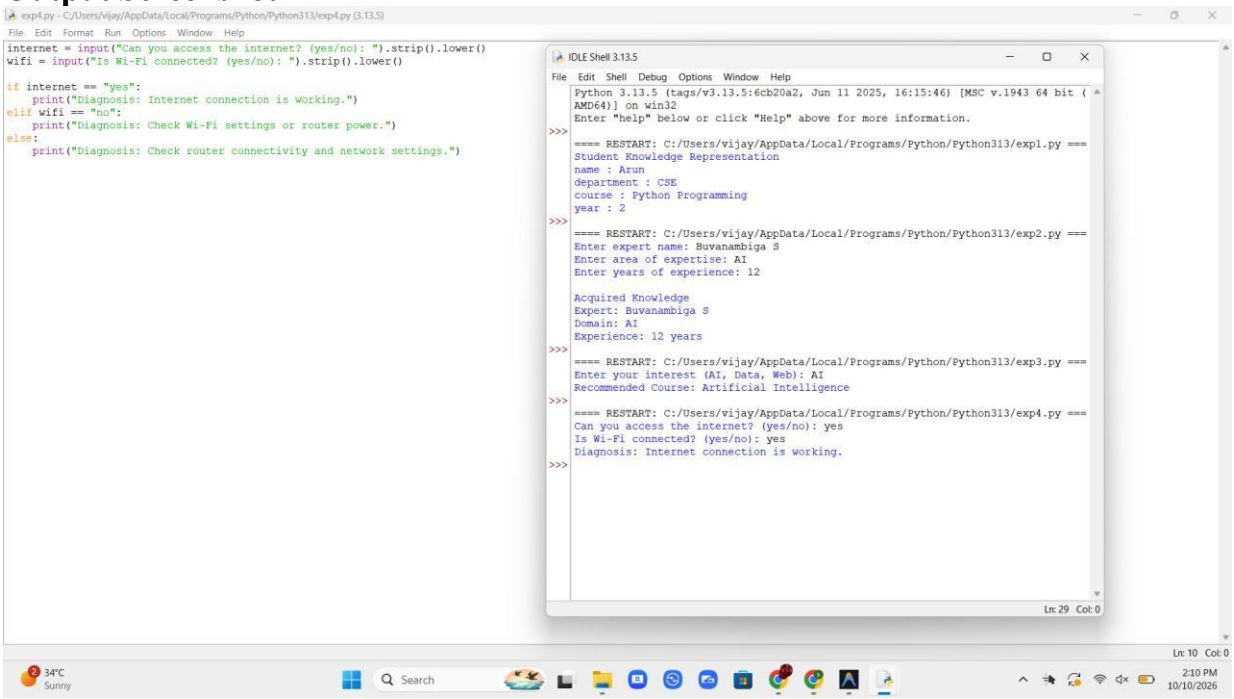
Program

```
print("Network Troubleshooting Expert System")
router_light = input("Is the router power light ON? (yes/no): ").strip().lower()
wifi = input("Is the device connected to Wi-Fi? (yes/no): ").strip().lower()
ping = input("Does ping to 8.8.8.8 succeed? (yes/no): ").strip().lower()
website = input("Do websites open in the browser? (yes/no): ").strip().lower()

if router_light == "no":
    advice = "Router has no power. Check the power cable and adapter."
elif wifi == "no":
    advice = "Device is not connected. Turn Wi-Fi on and re-enter the password."
elif ping == "no":
    advice = "No internet from ISP. Restart the router or call the provider."
elif website == "no":
    advice = "DNS problem. Change the DNS server to 8.8.8.8."
else:
    advice = "Network is working normally."

print("Diagnosis:", advice)
```

Output Screenshot



Result

A rule-based expert system was implemented that examined the symptoms entered by the user and displayed a suitable troubleshooting action. This is a teaching demonstration, not a professional diagnostic tool.

Experiment 5: Semantic Network Using NetworkX

Aim

To represent concepts and their labelled relationships as a semantic network and to infer inherited properties from it.

Software Required

Python 3.x (IDLE / any Python IDE) and NetworkX (pip install networkx)

Algorithm

1. **Start the program.** Import the networkx library as nx.
2. **Create a directed graph.** Use nx.DiGraph() so that every relationship has a direction from source concept to target concept.
3. **Add the is-a links.** Use add_edge() with relation="is-a" to connect Sparrow and Penguin to Bird, and Bird to Animal.
4. **Add property links.** Add the labelled edges Bird has Wings, Bird can Fly and Animal needs Food.
5. **Display the network.** Traverse G.edges(data=True) and print each relationship as source -- relation --> target.
6. **Infer inherited properties.** Starting from Sparrow, follow the is-a links upward. At each concept print the non-is-a relations, which Sparrow inherits from its parents.
7. **Stop the program.** The loop ends when a concept has no further parent.

Pseudocode

```
START
    CREATE directed graph G
    ADD edge Sparrow --is-a--> Bird
    ADD edge Penguin --is-a--> Bird
    ADD edge Bird --is-a--> Animal
    ADD edge Bird --has--> Wings
    ADD edge Bird --can--> Fly
    ADD edge Animal --needs--> Food
    FOR EACH edge (source, target, relation) IN G
        DISPLAY source, relation, target
    END FOR
    node <- "Sparrow"
    REPEAT
        FOR EACH non is-a edge OF node
            DISPLAY "Sparrow", relation, target
        END FOR
        node <- parent of node (is-a)
    UNTIL node has no parent
STOP
```

Program

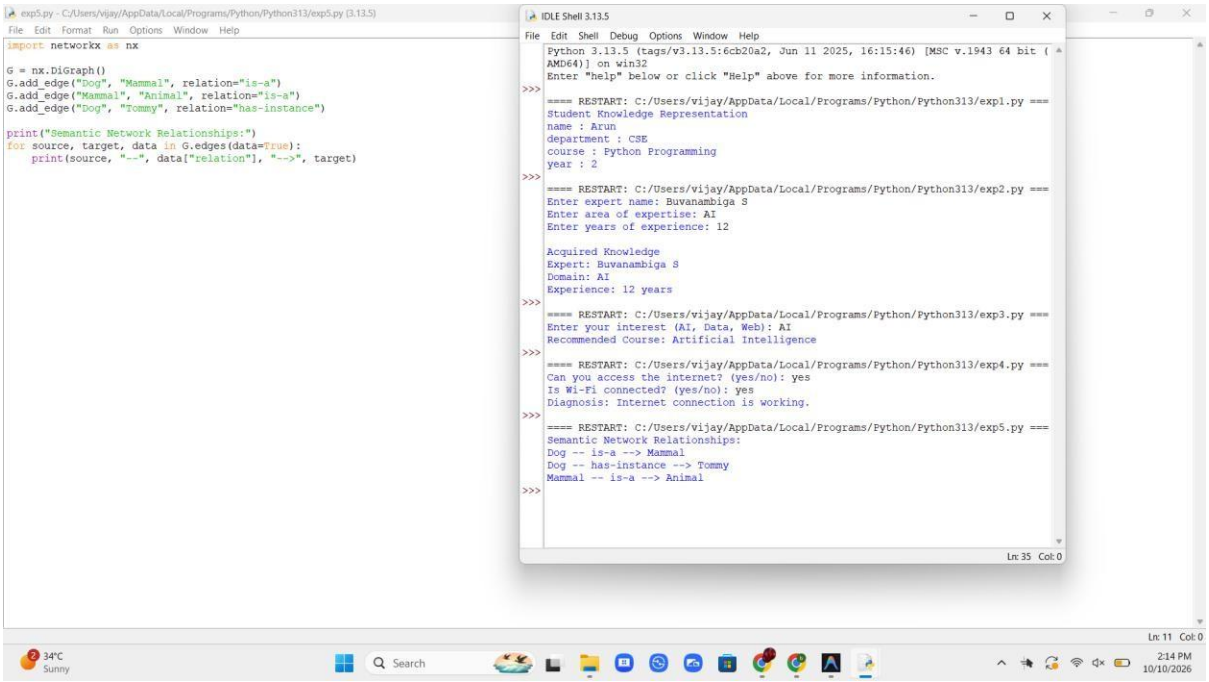
```
import networkx as nx

G = nx.DiGraph()
G.add_edge("Sparrow", "Bird", relation="is-a")
G.add_edge("Penguin", "Bird", relation="is-a")
G.add_edge("Bird", "Animal", relation="is-a")
G.add_edge("Bird", "Wings", relation="has")
G.add_edge("Bird", "Fly", relation="can")
G.add_edge("Animal", "Food", relation="needs")

print("Semantic Network Relationships:")
for source, target, data in G.edges(data=True):
    print(source, "--", data["relation"], "-->", target)

print("\nInheritance query for Sparrow:")
node = "Sparrow"
while True:
    parents = [t for _, t, d in G.out_edges(node, data=True) if d["relation"] == "is-a"]
    for _, t, d in G.out_edges(node, data=True):
        if d["relation"] != "is-a":
            print("Sparrow", d["relation"], t, "(inherited from", node + ")")
    if not parents:
        break
    node = parents[0]
```

Output Screenshot



Result

The semantic network represented concepts and labelled relationships, and the inheritance query showed that Sparrow inherits the properties of Bird and Animal.

Experiment 6: Frame-Based Knowledge Representation

Aim

To represent objects as frames with slots and values, and to retrieve slot values through inheritance from parent frames.

Software Required

Python 3.x (IDLE / any Python IDE)

Algorithm

1. **Start the program.** Plan a three-level frame hierarchy: Vehicle, Car and MyCar.
2. **Create the general frame.** Create the dictionary vehicle with the slots wheels, engine and fuel, and set is_a to None.
3. **Create the specialised frame.** Create the dictionary car with the slots doors and seats, and link it to vehicle using the is_a slot.
4. **Create the instance frame.** Create my_car with the slots brand, color and fuel, and link it to car. The fuel slot overrides the default value of the parent.
5. **Define get_slot().** Search the given frame for the slot. If it is missing, move to the parent frame through is_a and repeat until the slot is found or no parent remains.
6. **Display the frame.** For every slot, print the value and mention the frame it was inherited from. Print "not defined" if no frame holds the slot.
7. **Stop the program.**

Pseudocode

```
START
    CREATE frame Vehicle (wheels, engine, fuel)
    CREATE frame Car (doors, seats) WITH parent Vehicle
    CREATE frame MyCar (brand, color, fuel) WITH parent Car
    FUNCTION get_slot(frame, slot)
        WHILE frame EXISTS
            IF slot IN frame THEN
                RETURN value, frame name
            END IF
            frame <- parent of frame
        END WHILE
        RETURN none
    END FUNCTION
    FOR EACH slot IN slot list
        value, owner <- get_slot(MyCar, slot)
        DISPLAY slot, value, owner
    END FOR
STOP
```

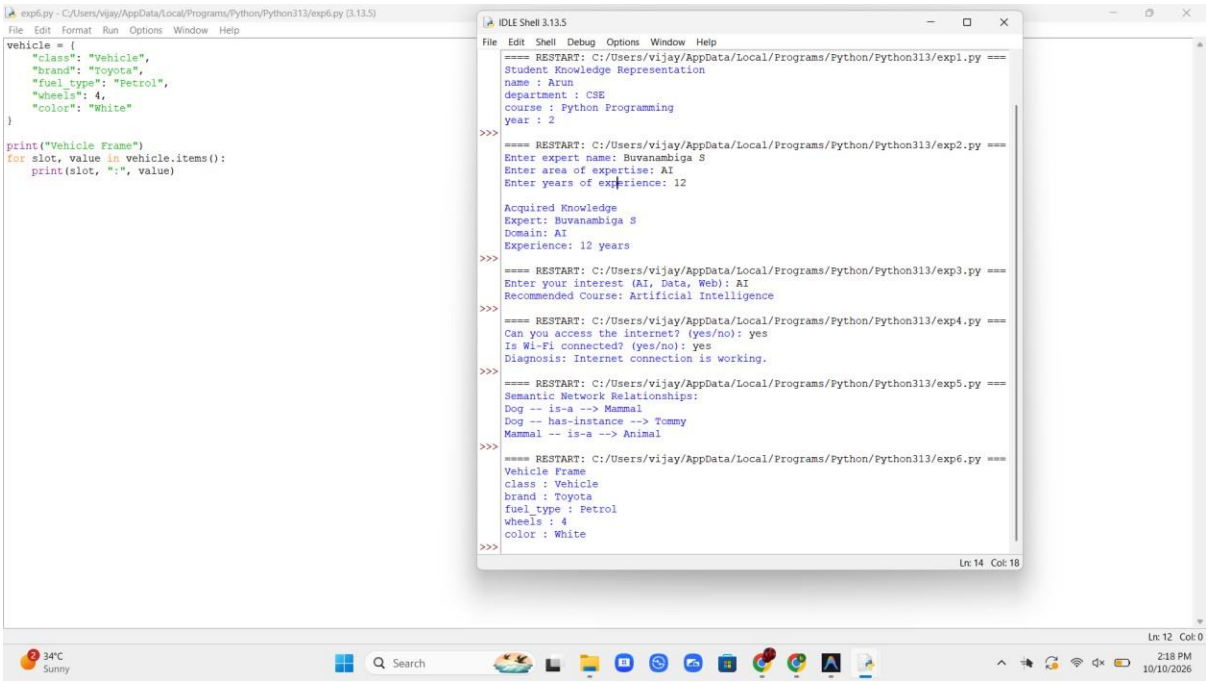
Program

```
vehicle = {"name": "Vehicle", "is_a": None, "wheels": 4, "engine": True, "fuel": "Petrol"}
car = {"name": "Car", "is_a": vehicle, "doors": 4, "seats": 5}
my_car = {"name": "MyCar", "is_a": car, "brand": "Honda", "color": "Red", "fuel": "Diesel"}

def get_slot(frame, slot):
    while frame is not None:
        if slot in frame:
            return frame[slot], frame["name"]
        frame = frame["is_a"]
    return None, None

print("Frame: MyCar")
for slot in ["brand", "color", "fuel", "doors", "seats", "wheels", "engine", "price"]:
    value, owner = get_slot(my_car, slot)
    if owner is None:
        print(slot, ": not defined")
    elif owner == "MyCar":
        print(slot, ":", value)
    else:
        print(slot, ":", value, "(inherited from", owner + ")")
```

Output Screenshot



Result

Objects were represented as frames with slots, and slot values were retrieved from parent frames by inheritance. The overriding value in the instance frame took priority over the default.

Experiment 7: Production Rules and Forward Chaining

Aim

To implement IF-THEN production rules and to derive new facts from known facts using forward chaining.

Software Required

Python 3.x (IDLE / any Python IDE)

Algorithm

1. **Start the program.** Decide the problem: finding the scholarship eligibility of a student.
2. **Initialise the facts.** Store marks_above_80, attendance_above_75 and family_income_low in a set named facts.
3. **Define the rules.** Store each rule as a name, a set of conditions and a conclusion, for example R1: marks and attendance give eligible_for_merit.
4. **Scan the rules.** For every rule, check whether all its conditions are present in facts using issubset().
5. **Fire a rule.** If the conditions are satisfied and the conclusion is new, add it to facts and print the rule name and the inferred fact.
6. **Repeat.** Keep scanning the rules until a full pass adds no new fact.
7. **Display the final facts and stop.** Print all facts in sorted order.

Pseudocode

```
START
    facts <- {marks_above_80, attendance_above_75, family_income_low}
    rules <- R1, R2, R3, R4 (conditions -> conclusion)
    REPEAT
        changed <- FALSE
        FOR EACH rule IN rules
            IF conditions OF rule ARE IN facts
               AND conclusion IS NOT IN facts THEN
                ADD conclusion TO facts
                DISPLAY rule name, conclusion
                changed <- TRUE
            END IF
        END FOR
    UNTIL changed = FALSE
    DISPLAY facts
STOP
```

Program

```
facts = {"marks_above_80", "attendance_above_75", "family_income_low"}

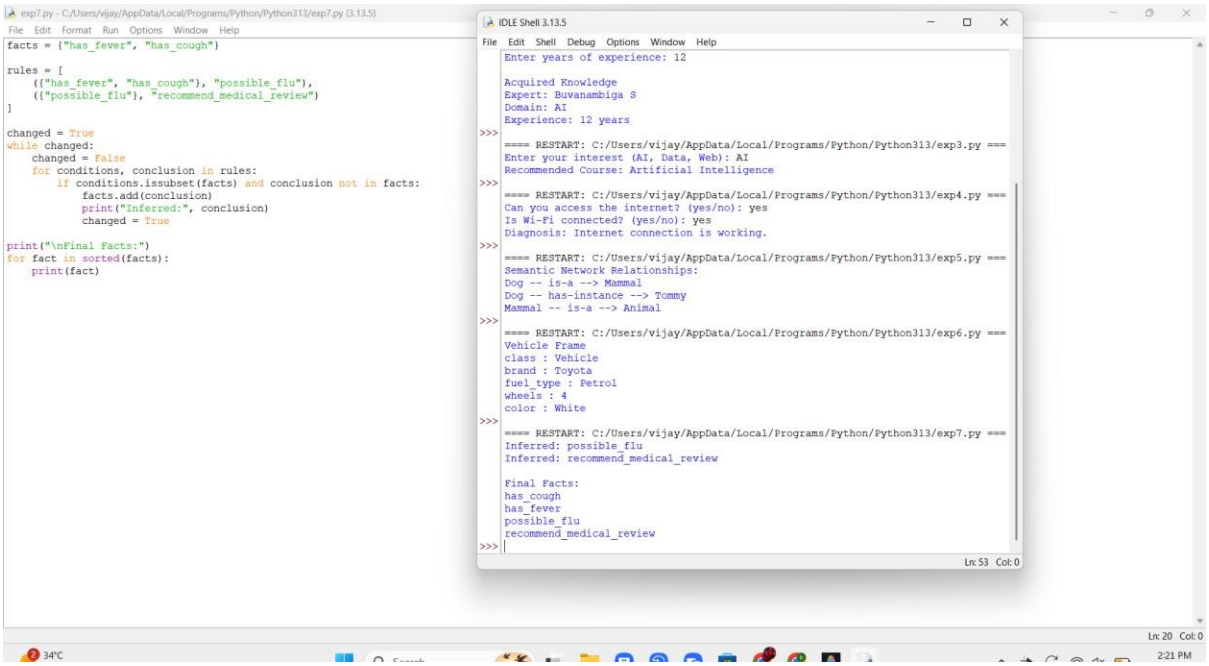
rules = [
    ("R1", {"marks_above_80", "attendance_above_75"}, "eligible_for_merit"),
    ("R2", {"family_income_low"}, "eligible_for_need_based"),
    ("R3", {"eligible_for_merit", "eligible_for_need_based"}, "full_scholarship"),
    ("R4", {"full_scholarship"}, "send_approval_letter"),
]

print("Initial Facts:", sorted(facts))
print()

step = 1
changed = True
while changed:
    changed = False
    for name, conditions, conclusion in rules:
        if conditions.issubset(facts) and conclusion not in facts:
            facts.add(conclusion)
            print("Step", step, ": Rule", name, "fired ->", conclusion)
            step += 1
            changed = True

print("\nFinal Facts:")
for fact in sorted(facts):
    print("-", fact)
```

Output Screenshot



Result

Forward chaining started with three known facts and derived four new facts by repeatedly applying the production rules. The scholarship rules are illustrative only.

Experiment 8: Ontology Development Using Protégé

Aim

To create a simple library ontology with classes, an object property and individuals, and to use a reasoner to infer new knowledge.

Software Required

Protégé Desktop 5.x with the HermiT reasoner

Algorithm

1. **Start Protégé.** Create a new ontology and set its IRI to http://example.org/LibraryOntology.
2. **Create the classes.** On the Classes tab add Person and Book under Thing.
3. **Create the subclasses.** Add Member and Librarian as subclasses of Person.
4. **Create the object property.** On the Object Properties tab add borrows, then set its Domain to Member and its Range to Book.
5. **Create the individuals.** On the Individuals tab add Meera (type Member), DataScienceBook (type Book), PythonBook (type Book) and Ravi (no type given).
6. **Assert the facts.** Add the object property assertions Meera borrows DataScienceBook and Ravi borrows PythonBook.
7. **Run the reasoner.** Select Reasoner > HermiT > Start reasoner.
8. **Inspect the inferred knowledge.** Select Ravi and read the inferred types, shown with a yellow background. Save the ontology as an .owl file.

Pseudocode

```
START
    CREATE ontology LibraryOntology
    CREATE classes Person, Book
    CREATE classes Member, Librarian
    Member SUBCLASS_OF Person
    Librarian SUBCLASS_OF Person
    CREATE object property borrows
        DOMAIN <- Member
        RANGE <- Book
    CREATE individual Meera OF TYPE Member
    CREATE individuals DataScienceBook, PythonBook OF TYPE Book
    CREATE individual Ravi (no type)
    ASSERT Meera borrows DataScienceBook
    ASSERT Ravi borrows PythonBook
    RUN reasoner
    DISPLAY inferred types OF Ravi
STOP
```

Implementation Steps

Classes:

- Thing
  - Person
    - Member
    - Librarian
  - Book

Object property:

- borrows (Domain: Member, Range: Book)

Individuals:

- Meera : Member
- DataScienceBook : Book
- PythonBook : Book
- Ravi : (no type asserted)

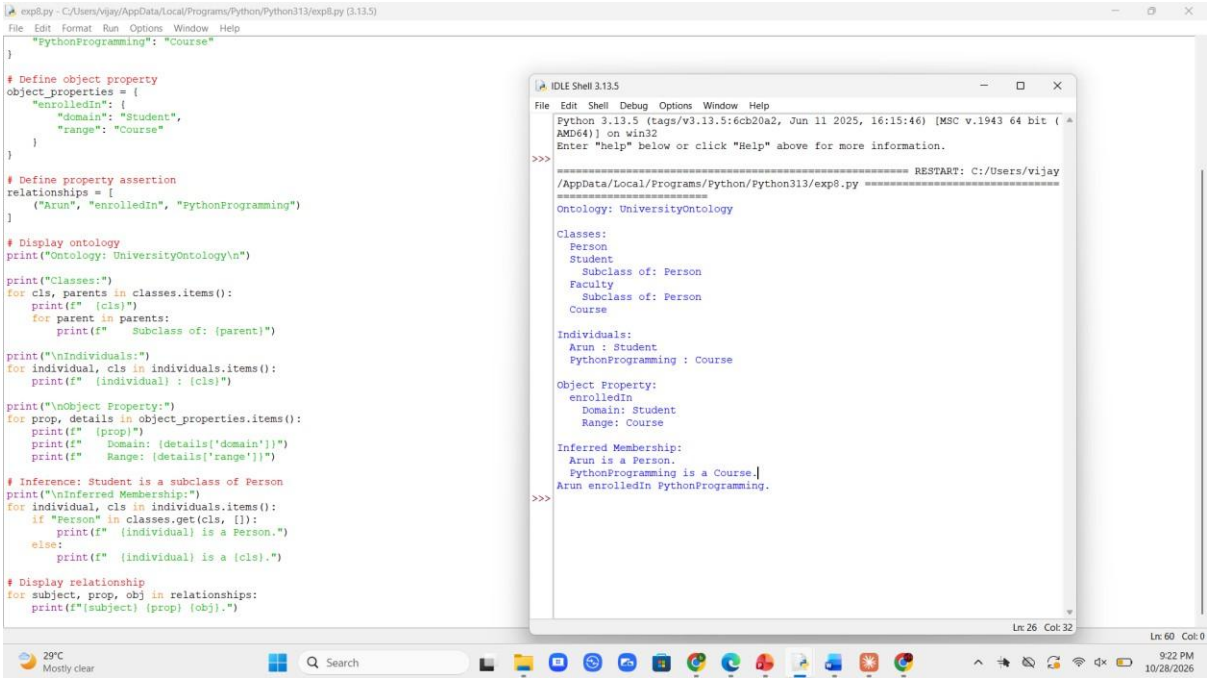
Property assertions:

- Meera borrows DataScienceBook
- Ravi borrows PythonBook

Expected inference after running HermiT:

- Ravi is a Member (from the domain of borrows)
- Ravi is a Person (Member is a subclass of Person)
- Meera is a Person (Member is a subclass of Person)

Output Screenshot



Result

A library ontology was created in Protégé, and the HermiT reasoner inferred that Ravi is a Member and a Person from the domain of the borrows property and the subclass relationships.

Experiment 9: Knowledge Graph Construction Using Python

Aim

To construct a knowledge graph of entities and relationships using a directed graph, and to query it.

Software Required

Python 3.x (IDLE / any Python IDE) and NetworkX (pip install networkx)

Algorithm

1. **Start the program.** Import the networkx library as nx.
2. **Create a directed graph.** Use nx.DiGraph() and store it in the variable kg.
3. **Add the director relationships.** Add edges with relation="directed" from Christopher Nolan to Inception and Interstellar.
4. **Add the actor relationships.** Add edges with relation="acted_in" from each actor to the movie he acted in.
5. **Add the genre relationships.** Add edges with relation="has_genre" from each movie to Sci-Fi.
6. **Display the triples.** Traverse kg.edges(data=True) and print each edge as (subject, relation, object).
7. **Display graph statistics.** Use number_of_nodes() and number_of_edges() to print the number of entities and relationships.
8. **Query the graph.** Read the out-edges of Christopher Nolan and print the movies whose relation is "directed". Then stop.

Pseudocode

```
START
    CREATE directed graph kg
    ADD (Christopher Nolan, directed, Inception)
    ADD (Christopher Nolan, directed, Interstellar)
    ADD (Leonardo DiCaprio, acted_in, Inception)
    ADD (Matthew McConaughey, acted_in, Interstellar)
    ADD (Inception, has_genre, Sci-Fi)
    ADD (Interstellar, has_genre, Sci-Fi)
    FOR EACH edge IN kg
        DISPLAY (subject, relation, object)
    END FOR
    DISPLAY number of nodes
    DISPLAY number of edges
    FOR EACH edge FROM "Christopher Nolan"
        IF relation = "directed" THEN
            DISPLAY movie
        END IF
    END FOR
STOP
```

Program

```
import networkx as nx

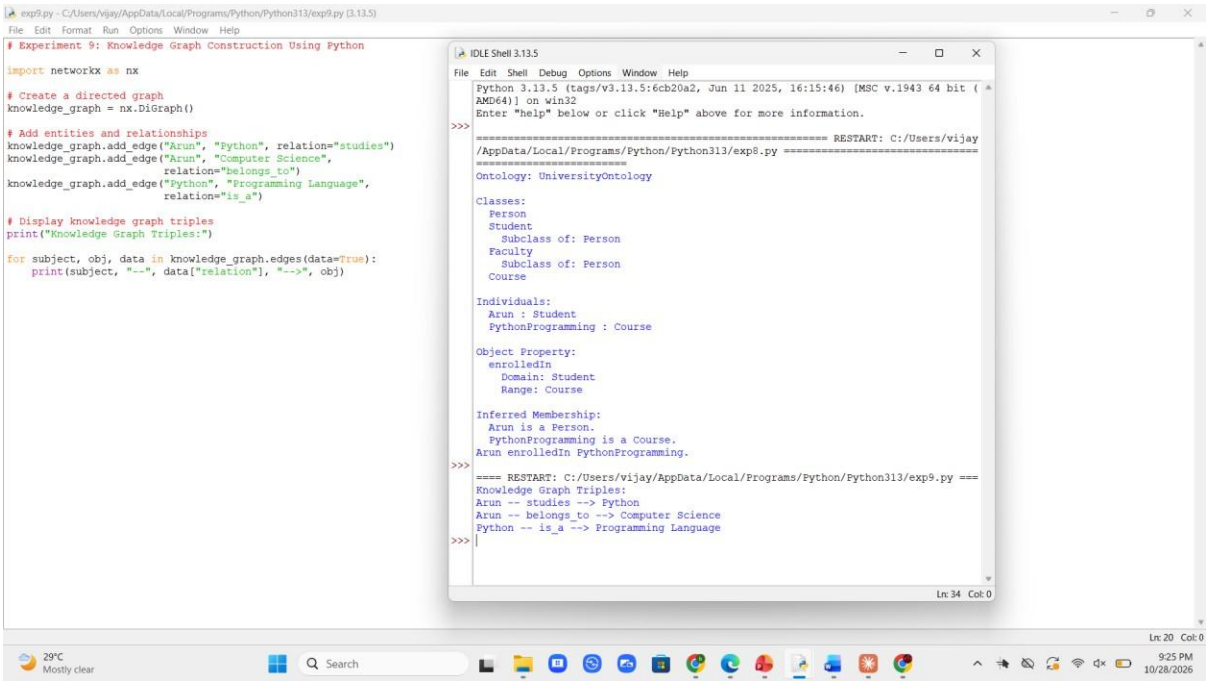
kg = nx.DiGraph()
kg.add_edge("Christopher Nolan", "Inception", relation="directed")
kg.add_edge("Christopher Nolan", "Interstellar", relation="directed")
kg.add_edge("Leonardo DiCaprio", "Inception", relation="acted_in")
kg.add_edge("Matthew McConaughey", "Interstellar", relation="acted_in")
kg.add_edge("Inception", "Sci-Fi", relation="has_genre")
kg.add_edge("Interstellar", "Sci-Fi", relation="has_genre")

print("Knowledge Graph Triples:")
for subject, obj, data in kg.edges(data=True):
    print("(", subject, ",", data["relation"], ",", obj, ")")

print("\nTotal entities:", kg.number_of_nodes())
print("Total relationships:", kg.number_of_edges())

print("\nQuery: Movies directed by Christopher Nolan")
for _, movie, data in kg.out_edges("Christopher Nolan", data=True):
    if data["relation"] == "directed":
        print("-", movie)
```

Output Screenshot



Result

A knowledge graph was constructed with entities as nodes and relationships as labelled edges, and a simple query retrieved the movies directed by a given person.

Experiment 10: Knowledge Management System

Aim

To build a menu-driven knowledge management system that can add, search and display knowledge items.

Software Required

Python 3.x (IDLE / any Python IDE)

Algorithm

1. **Start the program.** Create an empty dictionary named knowledge_base to hold topics and their details.
2. **Display the menu.** Inside a while loop show the options: add, search, display all and exit.
3. **Add knowledge.** If the choice is 1, read a topic and its details and store them in the dictionary.
4. **Search knowledge.** If the choice is 2, read a keyword and print every item whose topic or details contain it. Print a message if nothing matches.
5. **Display all knowledge.** If the choice is 3, print every topic with its details, or report that the knowledge base is empty.
6. **Handle invalid input.** For any other value except 4, print an error message and show the menu again.
7. **Exit.** If the choice is 4, print a message and leave the loop to stop the program.

Pseudocode

```
START
    CREATE empty dictionary knowledge_base
    REPEAT
        DISPLAY menu
        READ choice
        IF choice = 1 THEN
            READ topic, details
            knowledge_base[topic] <- details
        ELSE IF choice = 2 THEN
            READ keyword
            FOR EACH (topic, details) IN knowledge_base
                IF keyword IN topic OR details THEN
                    DISPLAY topic, details
                END IF
            END FOR
        ELSE IF choice = 3 THEN
            DISPLAY all items
        ELSE IF choice = 4 THEN
            EXIT loop
        ELSE
            DISPLAY "Invalid choice"
        END IF
    UNTIL choice = 4
STOP
```

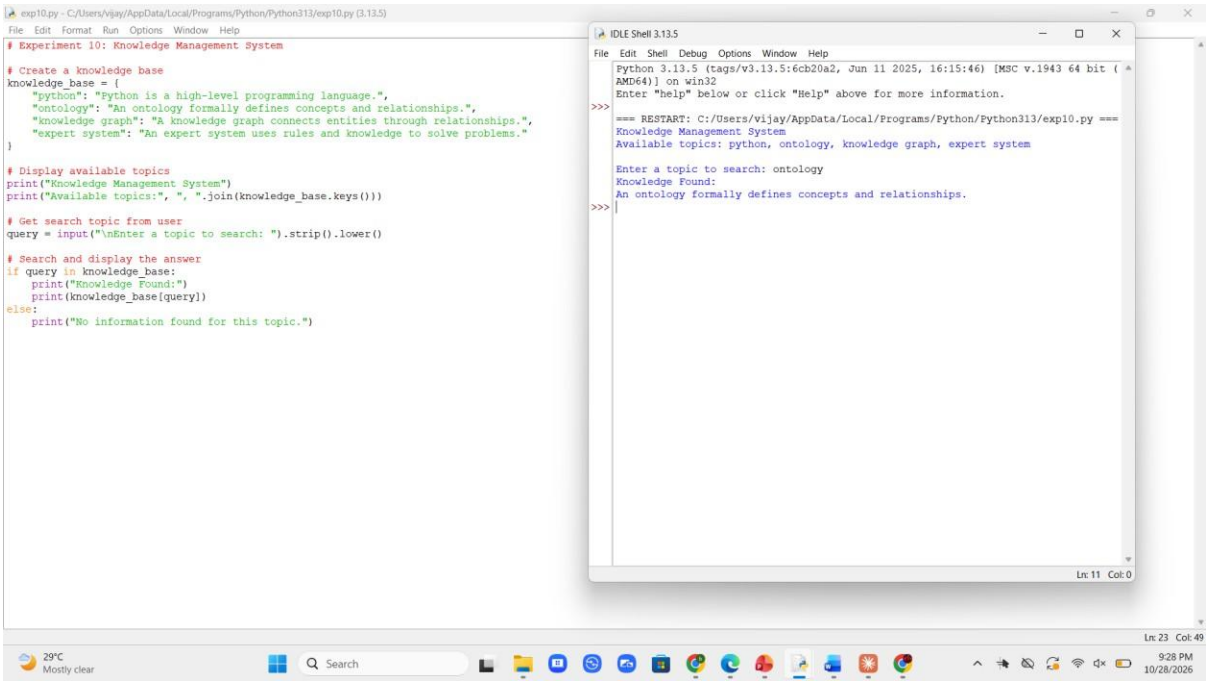
Program

```
knowledge_base = {}

while True:
    print("\n--- Knowledge Management System ---")
    print("1. Add knowledge")
    print("2. Search knowledge")
    print("3. Display all knowledge")
    print("4. Exit")
    choice = input("Enter your choice: ").strip()

    if choice == "1":
        topic = input("Enter topic: ").strip().lower()
        details = input("Enter details: ").strip()
        knowledge_base[topic] = details
        print("Knowledge stored successfully.")
    elif choice == "2":
        keyword = input("Enter keyword to search: ").strip().lower()
        found = False
        for topic, details in knowledge_base.items():
            if keyword in topic or keyword in details.lower():
                print(topic.title(), ":", details)
                found = True
        if not found:
            print("No matching knowledge found.")
    elif choice == "3":
        if not knowledge_base:
            print("Knowledge base is empty.")
        for topic, details in knowledge_base.items():
            print(topic.title(), ":", details)
    elif choice == "4":
        print("Exiting the system.")
        break
    else:
        print("Invalid choice. Try again.")
```

Output Screenshot



Result

A menu-driven knowledge management system was developed that captured, stored, searched and displayed knowledge items using a dictionary.

Experiment 11: RDF Knowledge Representation Using RDFLib

Aim

To represent facts as RDF triples, query them, and display the graph in Turtle format.

Software Required

Python 3.x (IDLE / any Python IDE) and RDFLib (pip install rdflib)

Algorithm

1. **Start the program.** Import Graph, Namespace, Literal and RDF from the rdflib library.
2. **Create the graph and namespace.** Create an empty Graph and the namespace EX = http://example.org/, then bind the prefix ex.
3. **Add type triples.** Use g.add() with RDF.type to state that Meera is a Student and Karthik is a Teacher.
4. **Add property triples.** Add the triples for name, studies, teaches and credits. Use Literal for plain values such as the name and credits.
5. **Display the triples.** Iterate over the graph in sorted order and print each subject, predicate and object using the ex prefix.
6. **Count the triples.** Print len(g) as the total number of triples.
7. **Query the graph.** Use g.subjects(EX.studies, EX.DataScience) to find who studies DataScience.
8. **Serialise the graph.** Use g.serialize(format="turtle") to print the graph in Turtle format, then stop.

Pseudocode

```
START
    CREATE graph g
    DEFINE namespace EX <- "http://example.org/"
    ADD (Meera, type, Student)
    ADD (Meera, name, "Meera")
    ADD (Meera, studies, DataScience)
    ADD (Karthik, type, Teacher)
    ADD (Karthik, teaches, DataScience)
    ADD (DataScience, credits, 4)
    FOR EACH (s, p, o) IN g
        DISPLAY s, p, o
    END FOR
    DISPLAY number of triples
    FIND subjects WHERE predicate = studies AND object = DataScience
    DISPLAY subjects
    SERIALISE g AS Turtle
    DISPLAY result
STOP
```

Program

```
from rdflib import Graph, Namespace, Literal, RDF

g = Graph()
EX = Namespace("http://example.org/")
g.bind("ex", EX)

g.add((EX.Meera, RDF.type, EX.Student))
g.add((EX.Meera, EX.name, Literal("Meera")))
g.add((EX.Meera, EX.studies, EX.DataScience))
g.add((EX.Karthik, RDF.type, EX.Teacher))
g.add((EX.Karthik, EX.teaches, EX.DataScience))
g.add((EX.DataScience, EX.credits, Literal(4)))

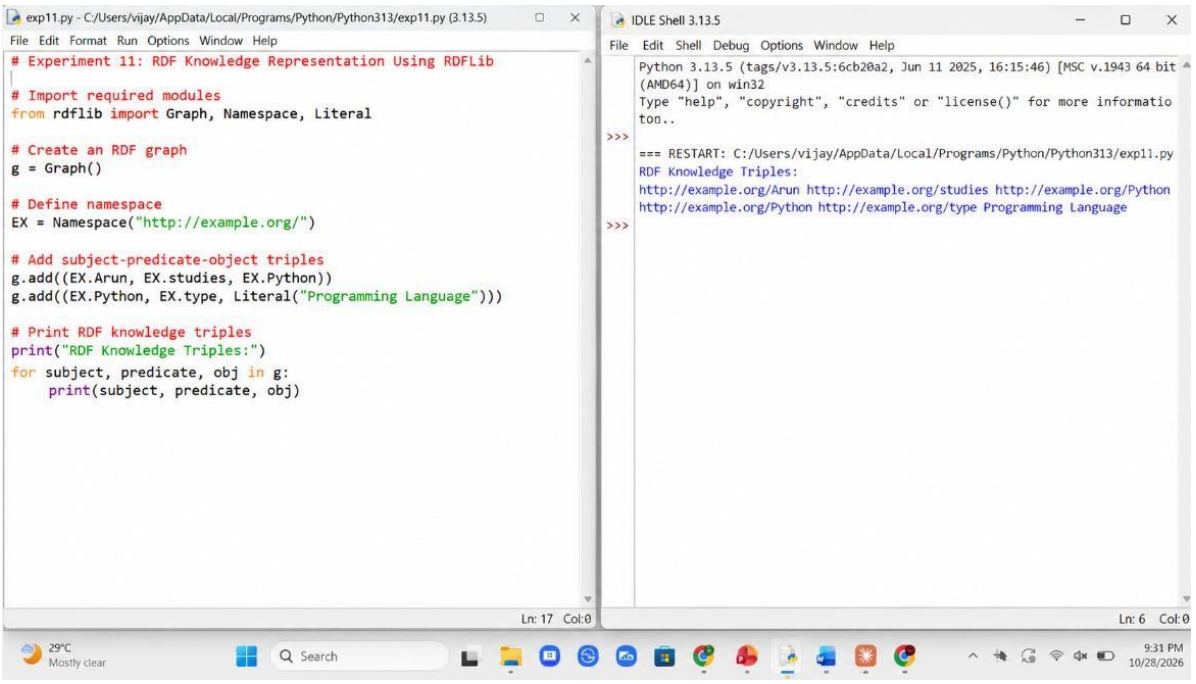
print("RDF Triples:")
for s, p, o in sorted(g):
    print(s.n3(g.namespace_manager), p.n3(g.namespace_manager),
    o.n3(g.namespace_manager))

print("\nTotal triples:", len(g))

print("\nQuery: Who studies DataScience?")
for s in g.subjects(EX.studies, EX.DataScience):
    print(s)

print("\nTurtle format:")
print(g.serialize(format="turtle"))
```

Output Screenshot



Result

Knowledge was represented as RDF subject-predicate-object triples, queried using RDFLib, and displayed in Turtle format. The order of the triples in the graph may vary.

Experiment 12: Knowledge Graph Traversal Using NetworkX

Aim

To traverse a knowledge graph using breadth-first search (BFS) and to find the shortest path between two entities.

Software Required

Python 3.x (IDLE / any Python IDE) and NetworkX (pip install networkx)

Algorithm

1. **Start the program.** Import networkx and deque from the collections module.
2. **Build the graph.** Create a graph with add_edges_from() linking concepts such as Meera, Data Science, Machine Learning and Deep Learning.
3. **Read the entities.** Accept the starting entity and the target entity from the user and check that both exist in the graph.
4. **Initialise BFS.** Put the path [start] in a queue and mark start as visited.
5. **Process the queue.** Remove the first path, take its last node and record the node in the visiting order.
6. **Check the target.** If the node is the target, return the path.
7. **Expand the node.** Otherwise add every unvisited neighbour to the visited set and push the extended path into the queue.
8. **Display the result and stop.** Print the BFS visiting order, the shortest path and its length, or a message if no path exists.

Pseudocode

```
START
    CREATE graph G, READ start, target
    IF start OR target NOT IN G THEN
        DISPLAY "Entity not found"
        STOP
    END IF
    queue <- [[start]], visited <- {start}
    WHILE queue IS NOT EMPTY
        path <- REMOVE first path FROM queue
        node <- last node OF path
        ADD node TO order
        IF node = target THEN
            DISPLAY order, path, length
            STOP
        END IF
        FOR EACH neighbour OF node
            IF neighbour NOT IN visited THEN
                ADD neighbour TO visited
                ADD (path + neighbour) TO queue
            END IF
        END FOR
    END WHILE
```

```
    DISPLAY "No path found"
STOP
```

Program

```
import networkx as nx
from collections import deque

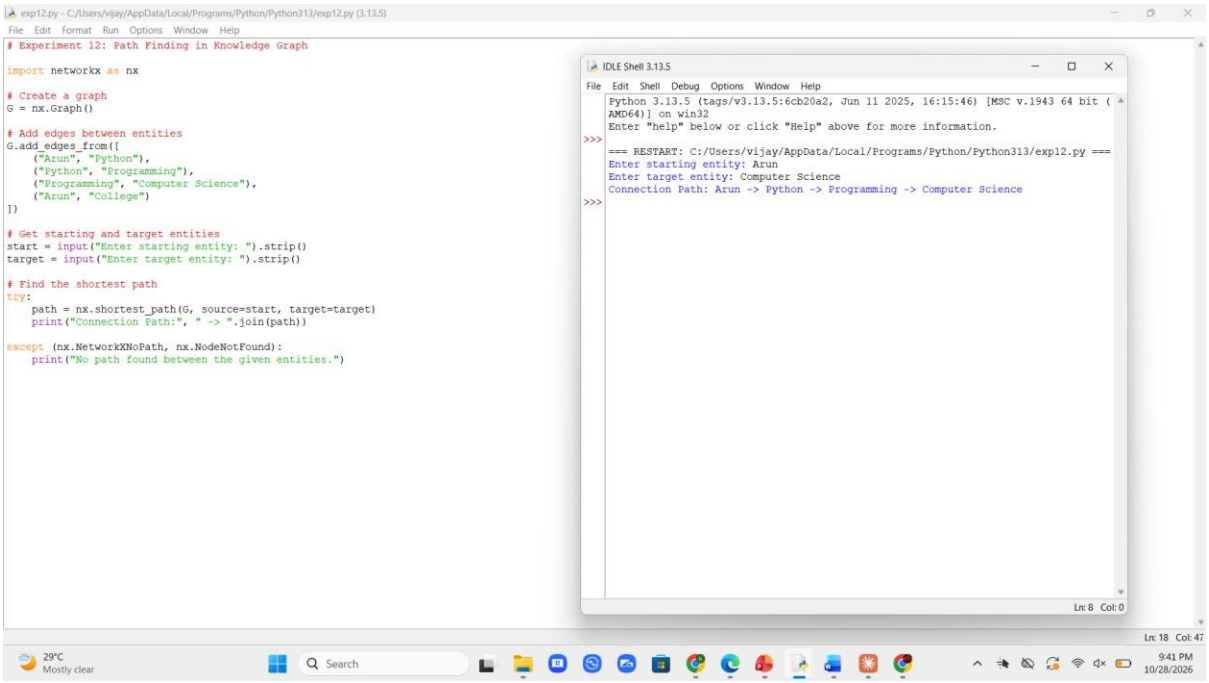
G = nx.Graph()
G.add_edges_from([
    ("Meera", "Data Science"),
    ("Data Science", "Machine Learning"),
    ("Machine Learning", "Neural Networks"),
    ("Data Science", "Statistics"),
    ("Meera", "Library"),
    ("Neural Networks", "Deep Learning"),
])

def bfs_path(graph, start, target):
    queue = deque([[start]])
    visited = {start}
    order = []
    while queue:
        path = queue.popleft()
        node = path[-1]
        order.append(node)
        if node == target:
            return path, order
        for neighbour in graph.neighbors(node):
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(path + [neighbour])
    return None, order

start = input("Enter starting entity: ").strip()
target = input("Enter target entity: ").strip()

if start not in G or target not in G:
    print("Entity not found in the knowledge graph.")
else:
    path, order = bfs_path(G, start, target)
    print("Nodes visited (BFS order):", " , ".join(order))
    if path:
        print("Shortest Path:", " -> ".join(path))
        print("Path length:", len(path) - 1, "edges")
    else:
        print("No path found.")
```

Output Screenshot



Result

Breadth-first search traversed the knowledge graph level by level and found the shortest connection between the two entities.